Exp: 14 Packet Sniffing using RAW Sockets

Aim:

To write a program using raw ~~sokcets~~ sockets to implement packet sniffing.

Requirements:

* Linux machine with Python 3
* Root privileges

Algorithm:

1. Create a raw packet socket that receives all Ethernet frames.
2. Receive a frame and unpack the 14-byte Ethernet header (destination MAC, source MAC, type).
3. For IPv4 frames unpack the IP header (source and destination IP, TTL, protocol, length).
4. Depending on the protocol unpack the TCP / UDP ports or the ICMP type and code.
5. Display the details of each captured packet.

Program (run: sudo python3 sniffer.py):

```
# packet sniffer using RAW sockets (Linux, run with sudo)
import socket, struct
s = socket.socket(socket.AF_PACKET, socket.SOCK_RAW, socket.ntohs(3))   # 3 = ETH_P_ALL
print("Sniffing... press Ctrl+C to stop\n")
mac = lambda b: ":".join("%02x" % x for x in b)
count = 0
try:
    while count < 20:
        raw, _ = s.recvfrom(65535)
        dst, src, proto = struct.unpack("!6s6sH", raw[:14])
        if proto != 0x0800:                          # only IPv4
            continue
        count += 1
```

```
            ip = struct.unpack("!BBHHHBBH4s4s", raw[14:34])
            ver_ihl, tlen, ttl, p, sip, dip = ip[0], ip[2], ip[5], ip[6], ip[8], ip[9]
            ihl = (ver_ihl & 0xF) * 4
            name = {1: "ICMP", 6: "TCP", 17: "UDP"}.get(p, str(p))
            print(f"Packet {count}: {mac(src)} -> {mac(dst)}")
            print(f"  IP : {socket.inet_ntoa(sip)} -> {socket.inet_ntoa(dip)}")
            print(f"       TTL={ttl} Protocol={name} Length={tlen}")
            t = raw[14+ihl:]
            if p == 6:
                sp, dp = struct.unpack("!HH", t[:4])
                print(f"  TCP : src port {sp} -> dst port {dp}")
            elif p == 17:
                sp, dp = struct.unpack("!HH", t[:4])
                print(f"  UDP : src port {sp} -> dst port {dp}")
            elif p == 1:
                print(f"  ICMP: type {t[0]} code {t[1]}")
    except KeyboardInterrupt:
        pass
```

Result:

The packet sniffer was implemented using raw sockets and the captured packet details were displayed.